

PetCare API

Diccionario de Errores y Reglas de Negocio

Versión 1.1 · Contrato de comportamiento y UX corregido

INTEGRARTEC · INTEGRADOR 4

Propósito del documento Definir cómo reacciona PetCare API ante entradas inválidas, fallas de autenticación, permisos insuficientes y conflictos de negocio. Establece códigos de error estables, mensajes de UX y reglas BR-* que deben mantenerse sincronizadas con el Documento 01.

Control del documento

Campo	Definición
Proyecto	PetCare API
Documento	02 — Diccionario de Errores y Reglas de Negocio
Versión	1.1
Estado	Baseline pre-implementación
Fecha	09/09/2026
Documento relacionado	01 — Especificación Funcional de la API (Contrato) v1.2
Próximo documento	03 — Diagramas de Flujo, Casos de Uso y Colección Postman

Cómo debe leerse este documento

El Documento 01 define qué ofrece la API y cómo se consume. Este Documento 02 define qué debe ocurrir cuando una operación no puede completarse y qué reglas del dominio deben respetarse antes de persistir cambios. El Documento 03 mostrará los flujos completos y sus pruebas operativas.

- El frontend debe tomar decisiones por el campo code y no comparando textos libres de message.
- statusCode expresa la categoría HTTP; code expresa el motivo estable de PetCare.
- message es un texto humano pensado para UX y soporte; no debe contener secretos ni detalles internos.
- Las reglas BR-* son fuente de verdad del comportamiento de negocio y deben traducirse a tests.
- Si una regla modifica campos, permisos o endpoints, también debe actualizarse el Documento 01.

Límite del MVP: no se crea una infraestructura de errores empresarial. Se define un contrato pequeño y consistente que puede implementarse con excepciones de NestJS, ValidationPipe y un mecanismo liviano de normalización de respuestas.

Índice funcional

1. Objetivo y alcance
2. Contrato general de errores
3. Criterio de códigos HTTP
4. Catálogo de códigos de error PetCare
5. Mensajes de validación
6. Reglas de negocio BR-*
7. Matriz detallada de permisos

8. Errores esperados por endpoint
9. Prioridad de resolución de errores
10. Guía de consumo para UX/frontend
11. Ejemplos canónicos de respuestas
12. Trazabilidad, tests y alineación con Integrador 4

1. Objetivo y alcance

Este documento permite que Product Owner, desarrollo y QA compartan una misma interpretación de los escenarios incorrectos de PetCare. Su objetivo es que la API falle de manera predecible, explicable y segura, sin depender de mensajes improvisados por cada endpoint.

1.1 Qué cubre

- Errores de validación de DTOs y parámetros.
- Autenticación: registro, login, access token, refresh y logout.
- Autorización sobre mascotas mediante PetAccess y roles OWNER, CAREGIVER y VIEWER.
- Conflictos de negocio al gestionar responsables compartidos.
- Errores del CRUD de Pet y Vaccination.
- Rate limiting y errores internos genéricos.
- Textos de mensajes canónicos para la primera versión.

1.2 Qué no cubre

- Errores de pagos, correo, almacenamiento de archivos o notificaciones, porque esas capacidades están fuera de V1.
- Códigos de organizaciones, adopciones, turnos veterinarios o historias clínicas.
- Internacionalización completa de mensajes; V1 utiliza español como copy de referencia.
- Observabilidad avanzada, trazas distribuidas o IDs de correlación.

1.3 Principio de diseño

Un error de API debe responder tres preguntas: qué categoría HTTP ocurrió, qué motivo estable puede interpretar el frontend y qué explicación humana puede mostrar o registrar.

2. Contrato general de errores

2.1 Estructura estándar

Todos los errores controlados de PetCare V1 deben tender a la siguiente estructura. details es opcional y se utiliza principalmente para validaciones de campos.

```
{
  "statusCode": 409,
  "code": "PET_LAST_OWNER",
  "error": "Conflict",
  "message": "La mascota debe conservar al menos un OWNER."
}
```

Campo	Tipo	Obligatorio	Contrato
statusCode	number	Sí	Código HTTP numérico de la respuesta.
code	string	Sí	Código estable de aplicación. El frontend puede usarlo para decidir el flujo.
error	string	Sí	Nombre estándar de la categoría HTTP: Bad Request, Unauthorized, Forbidden, etc.
message	string	Sí	Mensaje humano canónico. Nunca contiene stack trace, SQL, JWT, hashes o secretos.
details	array null	No	Detalle de campos inválidos cuando code = VALIDATION_ERROR.

2.2 Error de validación

```
{
  "statusCode": 400,
  "code": "VALIDATION_ERROR",
  "error": "Bad Request",
  "message": "Los datos enviados no son válidos.",
  "details": [
    { "field": "email", "message": "El email debe tener un formato válido." }
  ]
}
```

2.3 Estabilidad contractual

- code es estable dentro de PetCare V1 y no debe reutilizarse para otro significado.
- message puede corregirse editorialmente, pero un cambio que altere su significado debe actualizar este documento.
- El frontend no debe inferir permisos a partir del texto de message.
- Los errores inesperados se traducen a INTERNAL_ERROR y nunca exponen el error original en producción.

3. Criterio de códigos HTTP

HTTP	Nombre	Cuándo se usa en PetCare	Ejemplo
400	Bad Request	El request no cumple el contrato o contiene una combinación de datos inválida.	Fecha inválida, enum incorrecto, propiedad desconocida.
401	Unauthorized	No se pudo autenticar la sesión o las credenciales no son válidas.	JWT ausente/inválido; login incorrecto; refresh inválido.
403	Forbidden	El usuario está autenticado, pero no tiene acceso o su rol no permite la acción.	VIEWER intenta editar una vacunación.
404	Not Found	El recurso solicitado no existe dentro del contexto de la operación.	Pet o Vaccination inexistente.
409	Conflict	El request es válido, pero contradice el estado actual o una restricción de negocio.	Email duplicado; acceso duplicado; quitar último OWNER.
429	Too Many Requests	Se superó el límite configurado de solicitudes.	Demasiados intentos de login/register.
500	Internal Server Error	Falla inesperada no atribuible al request del cliente.	Error no controlado de infraestructura.

401 vs 403: 401 significa que la identidad no pudo validarse; 403 significa que la identidad sí es conocida, pero no tiene permiso suficiente. Para V1, si una mascota existe pero el usuario autenticado no posee PetAccess, se responde 403 PET_ACCESS_DENIED de forma consciente; no se oculta su existencia con 404.

4. Catálogo de códigos de error PetCare

4.1 Errores generales y validación

Código	HTTP	Mensaje canónico	Disparador
VALIDATION_ERROR	400	Los datos enviados no son válidos.	DTO o parámetros no cumplen el contrato.
INVALID_IDENTIFIER	400	El identificador enviado no es válido.	petId, userId o vaccinationId no tiene formato UUID válido.

Código	HTTP	Mensaje canónico	Disparador
RATE_LIMIT_EXCEEDED	429	Se realizaron demasiadas solicitudes. Intentá nuevamente más tarde.	Throttler bloquea temporalmente al cliente.
INTERNAL_ERROR	500	Ocurrió un error interno. Intentá nuevamente más tarde.	Excepción inesperada en producción.

4.2 Autenticación y usuarios

Código	HTTP	Mensaje canónico	Disparador
AUTH_EMAIL_IN_USE	409	El correo electrónico ya está registrado.	Registro con email existente.
AUTH_INVALID_CREDENTIALS	401	Las credenciales ingresadas no son válidas.	Email inexistente o contraseña incorrecta en login.
AUTH_ACCESS_REQUIRED	401	Necesitás iniciar sesión para continuar.	Access token ausente, expirado o inválido en ruta privada.
AUTH_REFRESH_INVALID	401	La sesión ya no puede renovarse. Iniciá sesión nuevamente.	Refresh ausente, inválido, expirado, revocado o no coincide con el hash.
USER_NOT_FOUND	404	No se encontró un usuario registrado con ese correo electrónico.	OWNER intenta compartir con un email sin cuenta PetCare.

Seguridad de login: AUTH_INVALID_CREDENTIALS usa el mismo mensaje para “email inexistente” y “contraseña incorrecta”; así no se confirma públicamente qué emails están registrados.

4.3 Mascotas y accesos compartidos

Código	HTTP	Mensaje canónico	Disparador
PET_NOT_FOUND	404	No se encontró la mascota solicitada.	petId válido pero inexistente.
PET_ACCESS_DENIED	403	No tenés acceso a esta mascota.	Usuario autenticado sin PetAccess para la mascota.
PET_ROLE_FORBIDDEN	403	Tu rol no permite realizar esta acción sobre la mascota.	Rol existente pero insuficiente.

Código	HTTP	Mensaje canónico	Disparador
PET_INVALID_BIRTH_DATE	400	La fecha de nacimiento no puede ser futura.	birthDate posterior a la fecha actual.
PET_ACCESS_EXISTS	409	El usuario ya tiene acceso a esta mascota.	Se intenta duplicar la relación userId + petId.
PET_ACCESS_NOT_FOUND	404	No se encontró el acceso solicitado para esta mascota.	Se intenta modificar/eliminar un PetAccess inexistente.
PET_LAST_OWNER	409	La mascota debe conservar al menos un OWNER.	Se intenta eliminar o degradar al último OWNER.

4.4 Vacunaciones

Código	HTTP	Mensaje canónico	Disparador
VACCINATION_NOT_FOUND	404	No se encontró la vacunación solicitada para esta mascota.	vaccinationId no existe o no pertenece al petId de la ruta.
VACCINATION_INVALID_DATES	400	Las fechas de vacunación no son válidas.	appliedAt es futura o nextDueAt < appliedAt, incluido el estado final de un PATCH.

5. Mensajes de validación

Las validaciones de forma pertenecen a los DTOs y al ValidationPipe. Para que UX y QA tengan un contrato estable, PetCare V1 define los siguientes textos canónicos. Cuando existan varios errores, details puede contener más de un elemento.

5.1 Auth

Campo	Condición	Mensaje
name	faltante o vacío	El nombre es obligatorio.
name	longitud fuera de 2-80	El nombre debe tener entre 2 y 80 caracteres.
email	faltante o vacío	El email es obligatorio.
email	formato inválido	El email debe tener un formato válido.
password	faltante o vacío	La contraseña es obligatoria.

Campo	Condición	Mensaje
password	longitud fuera de 8-72	La contraseña debe tener entre 8 y 72 caracteres.
refreshToken	faltante o vacío en refresh/logout	El refresh token es obligatorio.

5.2 Pet

Campo	Condición	Mensaje
name	faltante o vacío	El nombre de la mascota es obligatorio.
name	longitud fuera de 2-80	El nombre de la mascota debe tener entre 2 y 80 caracteres.
species	valor no permitido	La especie debe ser DOG, CAT u OTHER.
breed	más de 80 caracteres	La raza no puede superar los 80 caracteres.
birthDate	formato de fecha inválido	La fecha de nacimiento debe usar el formato YYYY-MM-DD.
careMode	valor no permitido	El modo de cuidado debe ser FAMILY o FOSTER.
rescueOrganizationName	más de 120 caracteres	El nombre de la organización no puede superar los 120 caracteres.

5.3 PetAccess

Campo	Condición	Mensaje
email	faltante o formato inválido	Ingresa un email válido para compartir el acceso.
role	valor no permitido	El rol debe ser OWNER, CAREGIVER o VIEWER.

5.4 Vaccination

Campo	Condición	Mensaje
vaccineName	faltante o vacío	El nombre de la vacuna es obligatorio.

Campo	Condición	Mensaje
vaccineName	longitud fuera de 2–120	El nombre de la vacuna debe tener entre 2 y 120 caracteres.
appliedAt	faltante	La fecha de aplicación es obligatoria.
appliedAt	formato inválido	La fecha de aplicación debe usar el formato YYYY-MM-DD.
appliedAt	fecha futura	La fecha de aplicación no puede ser futura.
nextDueAt	formato inválido	La próxima fecha debe usar el formato YYYY-MM-DD.
veterinarianName	más de 120 caracteres	El nombre del profesional no puede superar los 120 caracteres.
clinicName	más de 120 caracteres	El nombre del centro no puede superar los 120 caracteres.
notes	más de 1000 caracteres	Las notas no pueden superar los 1000 caracteres.

5.5 Propiedades no declaradas

```
{
  "statusCode": 400,
  "code": "VALIDATION_ERROR",
  "error": "Bad Request",
  "message": "Los datos enviados no son válidos.",
  "details": [
    { "field": "isAdmin", "message": "La propiedad 'isAdmin' no está permitida." }
  ]
}
```

Implementación esperada: ValidationPipe global con whitelist: true, forbidNonWhitelisted: true y transform: true, tal como exige el Integrador 4.

6. Reglas de negocio BR-*

Las reglas siguientes son comportamiento obligatorio de PetCare V1. “Capa principal” indica dónde conviene garantizar la regla, sin impedir que exista defensa en profundidad en otras capas.

6.1 Autenticación y sesión

ID	Regla	Capa principal	Resultado / error
BR-AUTH-001	El email de una cuenta es único. Se normaliza a lowercase antes de consultar/persistir.	DTO + Service + DB UNIQUE	Duplicado → AUTH_EMAIL_IN_USE (409).
BR-AUTH-002	La contraseña nunca se persiste en texto plano ni se devuelve en respuestas.	AuthService / mapper	Hash bcrypt costo 10–12. Sin error público específico.
BR-AUTH-003	Login no revela si falló el email o la contraseña.	AuthService	AUTH_INVALID_CREDENTIALS (401).
BR-AUTH-004	Toda ruta privada requiere access token válido.	JwtAuthGuard	AUTH_ACCESS_REQUIRED (401).
BR-AUTH-005	Refresh y logout reciben refreshToken en JSON y solo funcionan si el JWT es válido y coincide con refreshTokenHash. Logout deja el hash en null.	AuthService + DB	AUTH_REFRESH_INVALID (401).
BR-AUTH-006	V1 mantiene una sola sesión refresh activa por usuario. Un nuevo login reemplaza el hash anterior y /refresh no rota el refresh token.	AuthService + DB	El refresh anterior pasa a AUTH_REFRESH_INVALID (401).
BR-AUTH-007	Logout revoca la posibilidad de renovar sesión, pero no invalida retroactivamente un access JWT ya emitido.	JWT stateless + cliente	El cliente descarta tokens; el access puede vivir hasta su expiración (~15 min).

Decisión de alcance V1: no se crea una tabla Session por dispositivo ni una blacklist de access tokens. Hay un único refresh activo por usuario, no hay rotación en /refresh y logout revoca ese refresh. El access JWT ya emitido puede seguir siendo válido hasta expirar.

6.2 Mascotas

ID	Regla	Capa principal	Resultado / error
BR-PET-001	Crear una mascota y asignar al creador como OWNER forman una única operación de negocio.	PetsService + Prisma transaction/nested write	Si falla una escritura, se revierte todo.
BR-PET-002	Una mascota puede tener múltiples usuarios relacionados mediante PetAccess.	Modelo Prisma / DB	Relación N:M explícita.

ID	Regla	Capa principal	Resultado / error
BR-PET-003	Solo usuarios con PetAccess pueden consultar una mascota.	Service / autorización	PET_ACCESS_DENIED (403).
BR-PET-004	OWNER y CAREGIVER pueden editar datos básicos; VIEWER solo consulta.	Service / autorización	PET_ROLE_FORBIDDEN (403).
BR-PET-005	Toda mascota debe conservar al menos un OWNER. Las operaciones que reducen OWNER se ejecutan en transacción serializable; ante conflicto concurrente se reintenta o falla sin violar la invariante.	Service + Prisma transaction	PET_LAST_OWNER (409) o retry técnico.
BR-PET-006	Solo OWNER puede eliminar una mascota. Al eliminarla se eliminan PetAccess y Vaccination dependientes.	Service + relaciones Prisma	Rol insuficiente → 403; borrado válido → 204.
BR-PET-007	birthDate, si existe, no puede ser futura.	PetsService	PET_INVALID_BIRTH_DATE (400).
BR-PET-008	Todos los OWNER tienen autoridad equivalente en V1, incluida la eliminación definitiva de la mascota. No existe PRIMARY_OWNER.	Autorización / alcance	Sin error adicional.

6.3 Accesos compartidos

ID	Regla	Capa principal	Resultado / error
BR-ACCESS-001	Un usuario puede tener un solo PetAccess por mascota.	DB UNIQUE(userId, petId) + Service	PET_ACCESS_EXISTS (409).
BR-ACCESS-002	En V1 solo puede agregarse un usuario que ya tenga una cuenta PetCare.	PetAccessService	USER_NOT_FOUND (404).
BR-ACCESS-003	Solo un OWNER puede listar, agregar, cambiar o quitar accesos de la mascota.	Autorización	PET_ROLE_FORBIDDEN (403).
BR-ACCESS-004	Un OWNER puede modificar su propio rol o quitarse solo si queda al menos otro OWNER.	Service + transacción	PET_LAST_OWNER (409) si queda en cero.
BR-ACCESS-005	Los roles técnicos válidos son OWNER, CAREGIVER y VIEWER. Relaciones humanas no crean roles nuevos.	DTO / modelo	Valor inválido → VALIDATION_ERROR (400).

6.4 Vacunaciones

ID	Regla	Capa principal	Resultado / error
BR-VACC-001	Una vacunación siempre pertenece a una mascota existente.	VaccinationsService + FK	PET_NOT_FOUND (404) al crear sobre mascota inexistente.
BR-VACC-002	OWNER y CAREGIVER pueden crear, editar y eliminar vacunaciones; VIEWER solo consulta.	Autorización	PET_ROLE_FORBIDDEN (403).
BR-VACC-003	appliedAt no puede ser futura. Si nextDueAt existe, debe ser igual o posterior a appliedAt.	VaccinationsService	VACCINATION_INVALID_DATES (400).
BR-VACC-004	En PATCH las reglas de fechas se validan sobre el estado final resultante de combinar el registro persistido con el DTO.	VaccinationsService	VACCINATION_INVALID_DATES (400).
BR-VACC-005	Una ruta /pets/{petId}/vaccinations/{vaccinationId} solo opera si la vacunación pertenece a ese petId.	Consulta Prisma por id + petId	VACCINATION_NOT_FOUND (404).
BR-VACC-006	PetCare registra información declarativa; no calcula calendarios clínicos ni prescribe próximas dosis.	Alcance funcional	Sin error; evita lógica clínica fuera del MVP.

6.5 Seguridad y consistencia técnica

ID	Regla	Capa principal	Resultado / error
BR-SEC-001	Propiedades no declaradas en DTOs se rechazan.	ValidationPipe global	VALIDATION_ERROR (400).
BR-SEC-002	passwordHash, refreshTokenHash, JWT y secretos nunca se exponen en respuestas ni logs.	Mappers / logging	Sin error público; requisito de seguridad.
BR-SEC-003	register y login tienen rate limit reforzado.	@nestjs/throttler	RATE_LIMIT_EXCEEDED (429).
BR-SEC-004	Errores inesperados no exponen stack traces ni detalles internos en producción.	Exception handling	INTERNAL_ERROR (500).
BR-DATA-001	En PATCH: campo omitido conserva el valor; null limpia únicamente campos nullable; un valor reemplaza el existente.	DTO + Service	VALIDATION_ERROR (400) si null no está permitido.

ID	Regla	Capa principal	Resultado / error
BR-DATA-002	Las reglas cruzadas de negocio se evalúan sobre el estado final posterior al merge del registro existente con el DTO.	Service	Error de negocio específico, p. ej. VACCINATION_INVALID_DATES.

7. Matriz detallada de permisos

Acción	OWNER	CAREGIVER	VIEWER	Sin PetAccess
Ver datos básicos de la mascota	Sí	Sí	Sí	No (403)
Editar datos básicos	Sí	Sí	No (403)	No (403)
Eliminar mascota	Sí	No (403)	No (403)	No (403)
Listar responsables/accesos	Sí	No (403)	No (403)	No (403)
Agregar responsable	Sí	No (403)	No (403)	No (403)
Cambiar rol	Sí*	No (403)	No (403)	No (403)
Quitar responsable	Sí*	No (403)	No (403)	No (403)
Ver vacunaciones	Sí	Sí	Sí	No (403)
Crear vacunación	Sí	Sí	No (403)	No (403)
Editar vacunación	Sí	Sí	No (403)	No (403)
Eliminar vacunación	Sí	Sí	No (403)	No (403)

* Sujeto a BR-PET-005 / BR-ACCESS-004: la operación no puede dejar a la mascota sin OWNER.

“Responsable de respaldo”, “pareja”, “familiar” o “voluntario de ONG” son relaciones humanas. El permiso real depende del rol PetAccess asignado: OWNER, CAREGIVER o VIEWER.

8. Errores esperados por endpoint

8.1 Auth

Endpoint	Errores contractuales principales
POST /api/v1/auth/register	400 VALIDATION_ERROR · 409 AUTH_EMAIL_IN_USE · 429 RATE_LIMIT_EXCEEDED · 500 INTERNAL_ERROR
POST /api/v1/auth/login	400 VALIDATION_ERROR · 401 AUTH_INVALID_CREDENTIALS · 429 RATE_LIMIT_EXCEEDED · 500 INTERNAL_ERROR
POST /api/v1/auth/refresh	400 VALIDATION_ERROR · 401 AUTH_REFRESH_INVALID · 500 INTERNAL_ERROR
POST /api/v1/auth/logout	400 VALIDATION_ERROR · 401 AUTH_REFRESH_INVALID · 500 INTERNAL_ERROR
GET /api/v1/auth/me	401 AUTH_ACCESS_REQUIRED · 500 INTERNAL_ERROR

8.2 Pets

Endpoint	Errores contractuales principales
POST /api/v1/pets	400 VALIDATION_ERROR / PET_INVALID_BIRTH_DATE · 401 AUTH_ACCESS_REQUIRED · 500 INTERNAL_ERROR
GET /api/v1/pets	401 AUTH_ACCESS_REQUIRED · 500 INTERNAL_ERROR
GET /api/v1/pets/:petId	400 INVALID_IDENTIFIER · 401 AUTH_ACCESS_REQUIRED · 403 PET_ACCESS_DENIED · 404 PET_NOT_FOUND
PATCH /api/v1/pets/:petId	400 VALIDATION_ERROR / INVALID_IDENTIFIER / PET_INVALID_BIRTH_DATE · 401 · 403 PET_ACCESS_DENIED / PET_ROLE_FORBIDDEN · 404
DELETE /api/v1/pets/:petId	400 INVALID_IDENTIFIER · 401 · 403 PET_ACCESS_DENIED / PET_ROLE_FORBIDDEN · 404 PET_NOT_FOUND

8.3 PetAccess

Endpoint	Errores contractuales principales
GET /api/v1/pets/:petId/access	400 INVALID_IDENTIFIER · 401 · 403 PET_ACCESS_DENIED / PET_ROLE_FORBIDDEN · 404 PET_NOT_FOUND
POST /api/v1/pets/:petId/access	400 VALIDATION_ERROR / INVALID_IDENTIFIER · 401 · 403 · 404 PET_NOT_FOUND / USER_NOT_FOUND · 409 PET_ACCESS_EXISTS

Endpoint	Errores contractuales principales
PATCH /api/v1/pets/:petId/access/:userId	400 VALIDATION_ERROR / INVALID_IDENTIFIER · 401 · 403 · 404 PET_NOT_FOUND / PET_ACCESS_NOT_FOUND · 409 PET_LAST_OWNER
DELETE /api/v1/pets/:petId/access/:userId	400 INVALID_IDENTIFIER · 401 · 403 · 404 PET_NOT_FOUND / PET_ACCESS_NOT_FOUND · 409 PET_LAST_OWNER

8.4 Vaccinations

Endpoint	Errores contractuales principales
POST /api/v1/pets/:petId/vaccinations	400 VALIDATION_ERROR / INVALID_IDENTIFIER / VACCINATION_INVALID_DATES · 401 · 403 PET_ACCESS_DENIED / PET_ROLE_FORBIDDEN · 404 PET_NOT_FOUND
GET /api/v1/pets/:petId/vaccinations	400 INVALID_IDENTIFIER · 401 · 403 PET_ACCESS_DENIED · 404 PET_NOT_FOUND
GET /api/v1/pets/:petId/vaccinations/:vaccinationId	400 INVALID_IDENTIFIER · 401 · 403 PET_ACCESS_DENIED · 404 PET_NOT_FOUND / VACCINATION_NOT_FOUND
PATCH /api/v1/pets/:petId/vaccinations/:vaccinationId	400 VALIDATION_ERROR / INVALID_IDENTIFIER / VACCINATION_INVALID_DATES · 401 · 403 · 404 PET_NOT_FOUND / VACCINATION_NOT_FOUND
DELETE /api/v1/pets/:petId/vaccinations/:vaccinationId	400 INVALID_IDENTIFIER · 401 · 403 · 404 PET_NOT_FOUND / VACCINATION_NOT_FOUND

En las tablas anteriores “401” refiere a AUTH_ACCESS_REQUIRED y “403” se resuelve con el código específico indicado por la situación. Swagger debe documentar los status posibles; este documento conserva el motivo exacto.

9. Prioridad de resolución de errores

Una misma petición puede contener más de un problema. Para que tests, Swagger y UX observen resultados predecibles, se adopta el siguiente orden conceptual, alineado con el ciclo de NestJS:

1. Autenticación / Guard. Si una ruta privada no tiene una sesión válida, responde 401 antes de ejecutar el handler.
2. Pipes y validación de entrada. IDs, body y DTOs inválidos responden 400.
3. Existencia del recurso. Si el identificador es válido pero el recurso no existe, responde 404.
4. Autorización de recurso y rol. Si el usuario está autenticado pero no puede actuar sobre el recurso, responde 403.
5. Conflictos de estado. Si la operación sería válida salvo por el estado actual del negocio, responde 409.
6. Persistencia / infraestructura. Errores inesperados se convierten a 500 sin filtrar detalles internos.

Ejemplo: un VIEWER autenticado envía un PATCH con datos correctos sobre una mascota existente. La respuesta es 403 PET_ROLE_FORBIDDEN. Si no enviara JWT, la respuesta sería 401 antes de llegar a la validación de rol.

10. Guía de consumo para UX / frontend

10.1 Regla principal

El frontend decide qué hacer por code. message puede mostrarse al usuario, pero no debe utilizarse para ramificar lógica.

Código / familia	Comportamiento UX recomendado
VALIDATION_ERROR	Mantener el formulario abierto y asociar details a los campos correspondientes.
AUTH_INVALID_CREDENTIALS	Mostrar error general en login sin indicar si falló email o contraseña.
AUTH_ACCESS_REQUIRED	Intentar refresh una sola vez si el cliente dispone de refresh válido; si no, volver a login.
AUTH_REFRESH_INVALID	Limpiar sesión local y redirigir a login. También aplica si refresh/logout reciben un refreshToken expirado, revocado o reemplazado.
PET_ACCESS_DENIED / PET_ROLE_FORBIDDEN	Informar falta de permisos; no ocultar silenciosamente una acción ya intentada.
*_NOT_FOUND	Informar que el recurso ya no está disponible y actualizar la vista/listado.
PET_LAST_OWNER	Bloquear la acción y explicar que primero debe existir otro OWNER.
RATE_LIMIT_EXCEEDED	Deshabilitar temporalmente reintentos inmediatos y pedir esperar.
INTERNAL_ERROR	Mostrar mensaje genérico y permitir reintentar; nunca enseñar detalles técnicos.

10.2 Qué NO debería hacer el frontend

- Comparar literalmente message para decidir una navegación.
- Asumir que un 403 significa sesión vencida.

- Mostrar passwordHash, refreshTokenHash, token JWT, stack trace o mensajes SQL.
- Crear reglas distintas a las definidas por PetCare solo para “arreglar” un flujo del cliente.
- Reintentar automáticamente login/register en bucle cuando recibe 429.

11. Ejemplos canónicos de respuestas

11.1 Email duplicado — 409

```
{
  "statusCode": 409,
  "code": "AUTH_EMAIL_IN_USE",
  "error": "Conflict",
  "message": "El correo electrónico ya está registrado."
}
```

11.2 Request privado sin sesión — 401

```
{
  "statusCode": 401,
  "code": "AUTH_ACCESS_REQUIRED",
  "error": "Unauthorized",
  "message": "Necesitás iniciar sesión para continuar."
}
```

11.3 VIEWER intenta editar — 403

```
{
  "statusCode": 403,
  "code": "PET_ROLE_FORBIDDEN",
  "error": "Forbidden",
  "message": "Tu rol no permite realizar esta acción sobre la mascota."
}
```

11.4 Quitar al último OWNER — 409

```
{
  "statusCode": 409,
  "code": "PET_LAST_OWNER",
  "error": "Conflict",
  "message": "La mascota debe conservar al menos un OWNER."
}
```

11.5 Fechas de vacunación inválidas — 400

```
{
  "statusCode": 400,
  "code": "VACCINATION_INVALID_DATES",
  "error": "Bad Request",
  "message": "Las fechas de vacunación no son válidas.",
  "details": [
    { "field": "nextDueAt", "message": "La próxima fecha no puede ser anterior a la fecha de aplicación." }
  ]
}
```

11.6 Error interno — 500

```
{
  "statusCode": 500,
  "code": "INTERNAL_ERROR",
  "error": "Internal Server Error",
  "message": "Ocurrió un error interno. Intentá nuevamente más tarde."
}
```

12. Trazabilidad, tests y alineación con Integrador 4

12.1 Casos mínimos derivados de este documento

ID	Caso	Esperado
T-ERR-01	Registrar usuario con email inválido	400 VALIDATION_ERROR con details.email.
T-ERR-02	Registrar dos veces el mismo email	409 AUTH_EMAIL_IN_USE.
T-ERR-03	Login con credenciales incorrectas	401 AUTH_INVALID_CREDENTIALS.
T-ERR-04	Acceder a /pets sin JWT	401 AUTH_ACCESS_REQUIRED.
T-ERR-05	Usuario sin PetAccess consulta una mascota	403 PET_ACCESS_DENIED.
T-ERR-06	VIEWER intenta PATCH de Pet	403 PET_ROLE_FORBIDDEN.
T-ERR-07	OWNER agrega dos veces al mismo usuario	409 PET_ACCESS_EXISTS.
T-ERR-08	OWNER intenta quitar/degradar al último OWNER	409 PET_LAST_OWNER.
T-ERR-09	Crear vacunación con nextDueAt < appliedAt	400 VACCINATION_INVALID_DATES.
T-ERR-09B	Crear vacunación con appliedAt futura	400 VACCINATION_INVALID_DATES.
T-ERR-09C	PATCH cambia appliedAt y deja nextDueAt persistida en una combinación inválida	400 VACCINATION_INVALID_DATES sobre el estado final.

ID	Caso	Esperado
T-ERR-10	vaccinationId existe pero pertenece a otra mascota	404 VACCINATION_NOT_FOUND.
T-ERR-11	Enviar propiedad no declarada en DTO	400 VALIDATION_ERROR.
T-ERR-12	Superar límite de login/register	429 RATE_LIMIT_EXCEEDED.
T-ERR-13	Iniciar sesión dos veces y usar el primer refresh	401 AUTH_REFRESH_INVALID para el refresh anterior.
T-ERR-14	Logout con refresh vigente y luego intentar refresh con el mismo token	204 en logout y luego 401 AUTH_REFRESH_INVALID.
T-ERR-15	PATCH envía null en campo nullable y omite otro campo	El null limpia el campo y el omitido conserva su valor.

12.2 Relación con criterios del Documento 01

Criterio Doc. 01	Reglas / errores que lo respaldan
AC-01 / AC-02 / AC-03	BR-AUTH-001 a BR-AUTH-005; AUTH_*
AC-04	BR-AUTH-004; AUTH_ACCESS_REQUIRED
AC-05	BR-PET-001
AC-06 / AC-07 / AC-08 / AC-09	BR-PET-003 a BR-PET-006; BR-ACCESS-*; PET_*
AC-10	BR-PET-* y errores Pet
AC-11	BR-VACC-* y errores Vaccination
AC-12	BR-SEC-001; VALIDATION_ERROR / INVALID_IDENTIFIER
AC-13	BR-SEC-003 y controles de seguridad del Documento 01

12.3 Alineación con el Integrador 4

La consigna académica exige DTOs validados, rutas protegidas, autenticación access/refresh, seguridad básica y manejo consistente de errores. Este diccionario no agrega un nuevo dominio: operacionaliza esas exigencias y las reglas mínimas del modelo PetCare para que puedan implementarse y defenderse en el examen oral.

- ValidationPipe global: errores 400 predecibles y propiedades no declaradas rechazadas.
- AuthGuard y JWT: distinción consistente entre 401 y 403.
- Prisma/PostgreSQL: UNIQUE y relaciones respaldan conflictos y consistencia.

- Services: reglas que dependen del estado del negocio, como PET_LAST_OWNER y fechas de vacunación.
- Manejo de errores: ValidationPipe con exceptionFactory + ExceptionFilter global liviano para normalizar statusCode/code/error/message/details sin stack traces.
- Throttler: 429 y protección reforzada de register/login.

Definition of Done documental: ningún código listado aquí debe quedar sin implementación o sin test manual/automatizado equivalente, y Swagger debe documentar los status HTTP posibles por endpoint.

Decisiones congeladas para implementación

- Envelope de error: statusCode + code + error + message + details opcional; el frontend ramifica por code.
- Códigos de aplicación en inglés y mensajes canónicos de V1 en español.
- Roles cerrados a OWNER, CAREGIVER y VIEWER; no se crean permisos nuevos durante el MVP.
- V1 mantiene una única sesión refresh activa por usuario, no rota refresh en /refresh y no modela sesiones por dispositivo.
- Refresh y logout reciben { refreshToken } en JSON. Logout deja refreshTokenHash en null; el access JWT previo puede seguir válido hasta expirar.
- El último OWNER se protege con transacción serializable y 409 PET_LAST_OWNER; todos los OWNER tienen autoridad equivalente.
- PATCH usa semántica omitido=conservar, null=limpiar nullable, valor=reemplazar; las reglas cruzadas se validan sobre el estado final.
- Vaccination rechaza appliedAt futura y nextDueAt anterior a appliedAt; vacunas y rescate siguen siendo datos declarativos, sin lógica clínica.
- El contrato de error estable se implementará con ValidationPipe exceptionFactory y un ExceptionFilter global liviano; no se agregan códigos sin un escenario real y los 500 nunca exponen información interna.